

129. Un état de jeu

Quelques scènes, et un `switch` qui dit laquelle est en cours.

1. Ce qu'il faut comprendre

C'est presque toujours la forme d'un jeu : un `enum` de scènes, et un `switch`. Le reste — les ennemis, le décor — se branche dans le `case JEU`.

Le front est calculé **une fois**, en haut de la boucle, et servi à tous les cas. Le recalculer dans chaque `case` marcherait aussi — et un jour l'un des trois serait oublié.

Le score n'est écrit **que lorsqu'il change**, et l'étiquette une seule fois en entrant. Les réécrire à chaque image alourdit la boucle au point qu'elle déborde — et un `depuis` qui compte les tours avance alors trois fois moins vite que le temps réel.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **878 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage - la mise en route de la console
main()         208 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
LireManette    37 o ← bouton() - lit les huit boutons
Tuiles         336 o ← CopierTuiles - 21 lettres sur 43
"APPUIE START" 12 o ← texte() - un texte gravé
"              " 12 o ← texte() - un texte gravé
"SCORE"         5 o ← texte() - un texte gravé
"PERDU"         5 o ← texte() - un texte gravé
"              " 5 o ← texte() - un texte gravé
```

Lettres gravées : ACDEIOPRSTU0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 5    55 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
```

```
bouton() × 1    12 o sur place, dans main() + LireManette (37 o)
image()  × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois états : le TITRE (START pour jouer), le JEU (le score
// monte tout seul jusqu'à 20), puis PERDU (START pour revenir au titre).
//
// enum Scene ... : des noms pour les états : TITRE (0), JEU (1), PERDU (2).
// switch (ou) : on ne fait que ce qui concerne l'état actuel.
// Ce qui ne doit se faire qu'une fois (effacer, écrire SCORE) se fait
// au moment où l'on CHANGE d'état.

#include <bouton>    // lit un bouton de la manette
#include <texte>     // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. DES CONSTANTES NOMMÉES — SCENE

```
enum Scene { TITRE, JEU, PERDU };
```

3. UNE VARIABLE — OU

```
Scene ou = TITRE;           // L'état actuel.
```

4. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

5. UNE VARIABLE — AVANT

```
uint8_t avant = 0;          // START au tour d'avant.
```

6. UNE VARIABLE — DEPUIS

```
uint8_t depuis = 0;         // Les images depuis le dernier point.
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    while (true) {                          // Répète pour toujours :
        uint8_t appui = bouton(START);      // START maintenant ?
        uint8_t front = appui && !avant;     // 1 seulement au début de l'appui.
        avant = appui;

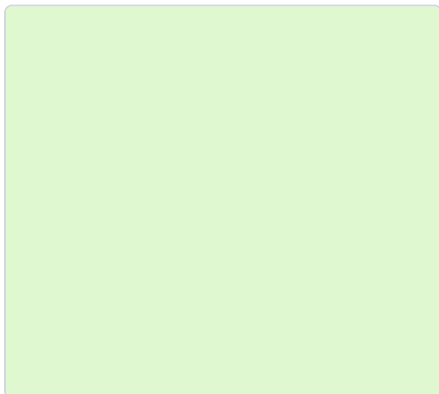
        switch (ou) {                      // Selon l'état :
            case TITRE:                    // --- le titre
                texte(4, 8, "APPUIE START");
                if (front) {                // START : on entre dans le jeu.
                    texte(4, 8, "          "); // efface le message (12 espaces),
                    score = 0; depuis = 0; ou = JEU; // remet à zéro, change d'état,
                    texte(2, 4, "SCORE");    // une fois, en entrant
                    nombre(9, 4, score);
                }
                break;

            case JEU:                      // --- le jeu
                depuis++;
                if (depuis >= 30) { depuis = 0; score++; nombre(9, 4, score); }
                // toutes les 30 images : un point.
                if (score >= 20) ou = PERDU; // à 20 points : fin.
                break;

            case PERDU:                    // --- la fin
                texte(5, 8, "PERDU");
                if (front) { texte(5, 8, "          "); ou = TITRE; } // START : retour au titre.
                break;
        }
        image();                          // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



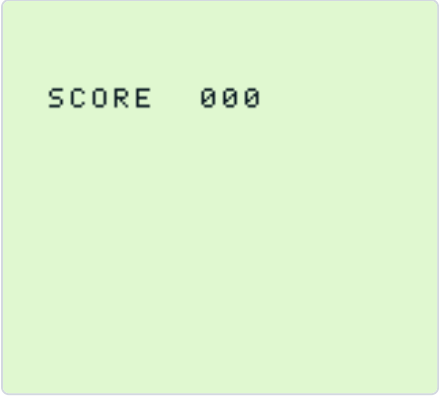
la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



SCORE 000

START — l'écran a changé

5. Ce qu'on doit voir

START lance la partie, le score monte, et à 20 c'est perdu.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre sur le titre
- ✓ START lance la partie (score 3)
- ✓ le score finit par atteindre vingt (20)
- ✓ et l'on passe à « PERDU »
- ✓ START ramène au titre

Ensuite : **129.1. La fonction `nombrePanneau()` — un nombre dans le panneau** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++